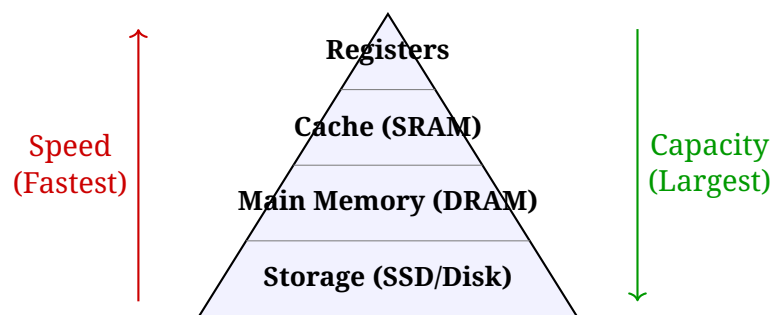

Advanced Computer Architecture

Academic Student Notes

UNIT - II: Principles of Scalable Performance & Hardware Technologies



Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management
(UGC Autonomous, R25 Regulation)

Compiled: July 14, 2026

Contents

1	Performance Metrics and Measures	2
1.1	Basic CPU Metrics	2
1.2	Common Performance Indexes	2
2	Parallel Processing Applications	2
3	Speedup Performance Laws	3
3.1	Amdahl's Law (Fixed Workload)	3
3.2	Gustafson's Law (Scaled Workload)	4
3.3	Sun and Ni's Law (Memory-Bounded Workload)	4
4	Scalability Analysis and Approaches	4
4.1	Isoefficiency Function	4
5	Hardware Technologies: Memory Hierarchy	5
5.1	SRAM vs. DRAM	5
6	Advanced Processor Technology	5
6.1	RISC vs. CISC Architectures	5
7	Superscalar and Vector Processors	6
7.1	Superscalar Processors	6
7.2	Vector Processors	6

1 Performance Metrics and Measures

To evaluate how well a computer executes programs, we need numeric indicators. Let's break down the basic metrics used to measure performance.

1.1 Basic CPU Metrics

- **Clock Cycle Time (τ):** The duration of one clock tick of the processor.
- **Clock Rate (f):** The frequency of the clock, equal to $f = 1/\tau$. (e.g., a 3.0 GHz processor ticks 3×10^9 times per second).
- **Instruction Count (I_c):** The total number of assembly language instructions executed by the program.
- **CPI (Cycles Per Instruction):** The average number of clock cycles required to execute a single instruction.

Note. Analogy: Imagine you are making sandwiches.

- **Instruction Count (I_c)** is the number of sandwiches you need to make.
- **CPI** is the average number of steps (cycles) you take to make one sandwich.
- **Clock Rate (f)** is how fast you move your hands (steps per second).

The total time (T) to execute a program is calculated using the fundamental equation:

$$T = I_c \times \text{CPI} \times \tau = \frac{I_c \times \text{CPI}}{f} \quad (1)$$

1.2 Common Performance Indexes

For marketing and general comparison, the industry uses two major indexes:

1. **MIPS (Million Instructions Per Second):** The rate of instruction execution per unit of time.

$$\text{MIPS} = \frac{I_c}{T \times 10^6} = \frac{f}{\text{CPI} \times 10^6}$$

Limitation: MIPS is not a fair comparison across different processor architectures (like RISC vs. CISC) because different processors require a different number of instructions to perform the exact same task.

2. **MFLOPS (Million Floating-Point Operations Per Second):** The rate of executing math operations (addition, multiplication) on decimal values.

$$\text{MFLOPS} = \frac{\text{Number of Floating-Point Operations}}{T \times 10^6}$$

This is highly relevant for scientific applications (simulations, AI model training) which rely heavily on decimal math.

2 Parallel Processing Applications

Why do we need parallel processing? Certain massive computational problems are simply impossible to solve on single-core systems in a reasonable timeframe.

- **Scientific Simulations:** Weather forecasting, climate modeling, and oceanography require solving complex physics equations across billions of grid points in 3D space.
- **Deep Learning & AI:** Training models (like GPT-4) requires performing billions of matrix multiplications. We use thousands of parallel GPU/TPU cores working simultaneously.
- **Bioinformatics:** DNA gene sequencing and protein folding simulation require comparing huge databases of molecular sequences.
- **Financial Engineering:** Monte Carlo simulations analyze thousands of market scenarios simultaneously to evaluate investment risks.

3 Speedup Performance Laws

When we add more processors to a computer, how much faster does it run? We measure this using **Speedup (S)**:

$$S_p = \frac{\text{Time taken by 1 processor}}{\text{Time taken by } p \text{ processors}} = \frac{T_1}{T_p}$$

Three major laws govern this relationship depending on how the workload scales.

3.1 Amdahl's Law (Fixed Workload)

Amdahl's Law assumes that the size of the program is fixed. If you add more processors, you want to solve the exact same problem faster.

Let f be the fraction of the program that can be executed in parallel. The remaining fraction $(1 - f)$ is sequential and must be run by only one processor.

Theorem 3.1 (Amdahl's Law)

The speedup S_p using p processors for a fixed workload is:

$$S_p = \frac{1}{(1 - f) + \frac{f}{p}} \quad (2)$$

Note. *The Traffic Analogy:* Imagine 80% of your journey is on the highway (parallel road where multiple lanes help flow) and 20% is stuck in a single-lane city street (sequential bottle-neck). Even if you buy a rocket car that travels down the highway at infinite speed ($p \rightarrow \infty$), your journey will still take at least the time you spend crawling through the city street ($1 - f = 20\%$). Your maximum possible speedup is $1/0.20 = 5\times$.

As the number of processors becomes infinitely large ($p \rightarrow \infty$), the speedup approaches a maximum limit:

$$\lim_{p \rightarrow \infty} S_p = \frac{1}{1 - f} \quad (3)$$

This tells us that **the sequential part of a program dictates the maximum speedup limit**, regardless of how many cores you throw at it.

3.2 Gustafson's Law (Scaled Workload)

Gustafson noticed that in the real world, users don't keep the problem size fixed when they buy faster computers. Instead, they scale up the problem size to get higher quality results in the same amount of time.

Let f be the parallel fraction of the scaled execution time on p processors.

Theorem 3.2 (Gustafson's Law)

The speedup S_p using p processors for a scaled workload (constant time) is:

$$S_p = p - (p - 1)(1 - f) = (1 - f) + p \cdot f \quad (4)$$

Note. *The Construction Analogy:* If you hire 10 workers, you don't just ask them to build one doghouse 10 times faster. Instead, you utilize their collective labor to build a whole house in the same week. Gustafson's Law represents a linear scaling of performance when the amount of work matches the size of the hardware.

3.3 Sun and Ni's Law (Memory-Bounded Workload)

Sun and Ni realized that the limiting factor in scaling workloads is often not time, but the physical memory capacity of the computer. When we add more nodes (each with its own memory), the total memory capacity increases, allowing us to run even larger algorithms.

Let W be the total workload. Sun and Ni's speedup equation adjusts the parallel workload fraction based on a scale factor $g(p)$, which represents how much the workload can expand given the extra memory:

$$S_p = \frac{(1 - f) + f \cdot g(p)}{(1 - f) + \frac{f \cdot g(p)}{p}} \quad (5)$$

- If $g(p) = 1$, it reduces to Amdahl's Law (workload cannot grow).
- If $g(p) = p$, it reduces to Gustafson's Law (workload grows linearly).
- If $g(p) > p$, the workload grows faster than the number of processors, leading to **superlinear speedup** in memory-bound applications.

4 Scalability Analysis and Approaches

Scalability measures the ability of a parallel system (hardware + algorithm) to deliver a proportional increase in speedup as we add more processors and scale the workload size.

4.1 Isoefficiency Function

If you add more processors to a system but keep the problem size the same, the efficiency drops because of communication overhead. To keep the efficiency (E) constant, you must increase the workload (W).

The **Isoefficiency Function** tells you exactly how fast the workload must grow relative to the number of processors (p) to maintain a steady efficiency rate:

- **Low Isoefficiency (e.g., $O(p)$):** The system is highly scalable. You only need to scale the workload slightly to keep efficiency high.
- **High Isoefficiency (e.g., $O(p^2)$ or $O(2^p)$):** The system scales poorly. You must make the workload massive just to keep up with the overhead of the extra cores.

5 Hardware Technologies: Memory Hierarchy

Computer memory is structured like a pyramid because of a fundamental trade-off: **faster memory is more expensive and smaller, while cheaper memory is larger but slower.**

Table 1: Memory Hierarchy Parameters

Level	Typical Technology	Access (Speed)	Time	Typical Capacity
Registers	Flip-Flops (on the CPU core)	< 1 ns		< 1 KB
Cache	SRAM (Static RAM)	1 – 10 ns		32 KB – 64 MB
Main Memory	DRAM (Dynamic RAM)	50 – 100 ns		8 GB – 128 GB
Secondary Storage	Flash Memory / SSD / HDD	0.1 – 10 ms		512 GB – 10 TB

5.1 SRAM vs. DRAM

- **SRAM (Static RAM):** Uses a 6-transistor latch design to store each bit of data. It holds data indefinitely as long as power is supplied. It is extremely fast but takes up a lot of chip space, making it expensive. Used for **Caches**.
- **DRAM (Dynamic RAM):** Uses only 1 transistor and 1 capacitor to store each bit. The capacitor slowly leaks charge, meaning it must be constantly refreshed thousands of times per second. It is slower than SRAM but much denser and cheaper. Used for **Main Memory**.

6 Advanced Processor Technology

Processors read instruction sets to run software. The design philosophy of these instruction sets split the industry into two camps:

6.1 RISC vs. CISC Architectures

- **RISC (Reduced Instruction Set Computer):** RISC focus is on simple, fast instructions that all take exactly 1 clock cycle to run. Heavy work is offloaded to the compiler, which combines simple instructions to execute complex tasks. *Example:* ARM (used in iPhones and Apple M-series chips), MIPS, RISC-V.
- **CISC (Complex Instruction Set Computer):** CISC focus is on complex instructions that do a lot of work in a single step (e.g., multiplying two numbers in memory directly). These take multiple cycles to execute, saving memory space but making

the hardware inside the processor highly complex. *Example:* Intel x86 and AMD processors.

Table 2: RISC vs. CISC Comparison

Feature	RISC	CISC
Instruction Size	Fixed length (usually 32-bit).	Variable length (1 to 15 bytes).
Addressing Modes	Very few (simplifies hardware decoding).	Many complex addressing modes.
Execution Speed	Most instructions take 1 clock cycle.	Variable clock cycles per instruction.
Registers	Large general-purpose register file.	Fewer registers (uses memory directly).
Compiler Role	Crucial (translates code into simple steps).	Simpler (hardware handles complexity).

7 Superscalar and Vector Processors

How do processors execute multiple tasks at once at the hardware level?

7.1 Superscalar Processors

A **superscalar processor** can issue and execute multiple independent instructions during a single clock cycle. It does this by copying arithmetic hardware units (like ALUs, Floating-Point Units) inside the core.

Analogy: Imagine a chef with four arms. Instead of doing one step at a time, the chef can chop carrots, stir soup, and boil water simultaneously in a single cycle.

At runtime, hardware logic scans the upcoming instructions, determines which ones do not depend on each other, and dispatches them to different execution units simultaneously.

7.2 Vector Processors

A **vector processor** is designed to execute a single instruction on an entire array of data values (a vector) in one go. Instead of running a loop to add 64 pairs of numbers, a vector processor loads the arrays into large vector registers and adds them all simultaneously using a pipeline.

Analogy: An assembly line conveyor belt. Once the pipeline is full, it outputs one finished calculation result every single clock cycle.

Vector processing eliminates the overhead of loop branch execution and instruction fetching, making it highly efficient for graphics, scientific math, and machine learning workloads.